

MYSQL 8.0

Module 1 — Student-Friendly SQL Notes

Custom Practice Database + Employees + Sakila

Beginner-first notes aligned to the supplied 12-hour session plan

Teaching approach: Learn the idea using a small, controlled database first. Then apply the same concept to the enterprise-style **employees** and **sakila** databases.

Environment: MySQL 8.0 + MySQL Workbench 8.0. The supplied session plan uses employees and sakila for demonstrations.

1. How to Use These Notes

These notes are built around the custom SQL command file. Students should not memorize SQL blindly. For every query, follow this cycle:

- **Read the comment** — understand the task.
- **Predict the result** — say what you think MySQL will return.
- **Run the query** — observe the actual result.
- **Change one part** — experiment safely.
- **Explain it in English** — describe what the query is doing.

The custom practice database is called **module1_lab**. It contains a simple **students** table and a **departments** table so relationships and JOINS can be understood without the complexity of the full sample databases.

2. SQL Command Categories

Category	Simple meaning	Commands
DDL	Build or change the database structure	CREATE, ALTER, DROP, TRUNCATE
DML	Add, change, or remove stored data	INSERT, UPDATE, DELETE, REPLACE
DQL	Ask the database for information	SELECT
TCL	Control whether changes are saved	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT
DCL	Control permissions	GRANT, REVOKE

Memory trick: DDL = design the structure; DML = manipulate data; DQL = question the data; TCL = control changes; DCL = control access.

3. Create the Practice Database

3.1 Create and select the database

```
CREATE DATABASE IF NOT EXISTS module1_lab;  
USE module1_lab;  
  
SELECT DATABASE();  
SHOW DATABASES;
```

Layman meaning: Create a practice cupboard called **module1_lab**, then tell MySQL that this is the cupboard we want to work inside.

3.2 Create the students table

```
DROP TABLE IF EXISTS students;  
  
CREATE TABLE students (  
  student_id INT PRIMARY KEY AUTO_INCREMENT,  
  student_name VARCHAR(100) NOT NULL,  
  age INT,  
  course VARCHAR(50),  
  marks INT,  
  email VARCHAR(150) UNIQUE,  
  status VARCHAR(20) DEFAULT 'Active'  
);
```

Think of CREATE TABLE as: “Build an empty form with these fields and rules.”

Column	Type / Rule	Meaning
student_id	INT, PRIMARY KEY, AUTO_INCREMENT	Unique student number generated by MySQL
student_name	VARCHAR(100), NOT NULL	Student's name; a value is required
age	INT	Whole-number age
course	VARCHAR(50)	Course name
marks	INT	Marks
email	VARCHAR(150), UNIQUE	Email; duplicate values are not allowed
status	VARCHAR(20), DEFAULT 'Active'	If omitted, MySQL uses Active

4. SELECT — Ask the Database

4.1 SELECT everything

```
SELECT *
FROM students;
```

Meaning: “Show me all columns and all rows from students.”

4.2 Select only what you need

```
SELECT student_name, marks
FROM students;
```

Meaning: “Show only the student's name and marks.”

4.3 AS — give the result a temporary friendly name

```
SELECT
  student_name AS Name,
  marks AS Score
FROM students;
```

AS changes the label in the result. It does not normally rename the actual table column.

4.4 WHERE — filter rows

```
SELECT *
FROM students
WHERE marks > 80;
```

Meaning: “Show only students whose marks are greater than 80.”

Operator	Meaning
=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

4.5 AND, OR, NOT

```
-- Both conditions must be true
SELECT *
FROM students
```

```

WHERE age >= 20
AND marks >= 80;

-- Either condition can be true
SELECT *
FROM students
WHERE marks >= 90
OR age = 19;

-- Reverse the condition
SELECT *
FROM students
WHERE NOT course = 'BCA';

```

4.6 DISTINCT

```

SELECT DISTINCT course
FROM students;

```

Meaning: Show each different course only once.

4.7 ORDER BY

```

SELECT student_name, marks
FROM students
ORDER BY marks DESC;

```

Meaning: Sort the result. **ASC** means low-to-high / A-to-Z; **DESC** means high-to-low / Z-to-A.

4.8 LIMIT

```

SELECT *
FROM students
LIMIT 3;

```

Meaning: Return only the first three rows of the result.

5. Counting and Calculating

5.1 COUNT, SUM, AVG, MIN, MAX

```

SELECT COUNT(*) AS total_students
FROM students;

SELECT SUM(marks) AS total_marks
FROM students;

SELECT AVG(marks) AS average_marks
FROM students;

SELECT MIN(marks) AS lowest_marks
FROM students;

SELECT MAX(marks) AS highest_marks
FROM students;

```

Function	Layman meaning
COUNT()	How many?
SUM()	What is the total?
AVG()	What is the average?
MIN()	What is the smallest?
MAX()	What is the largest?

5.2 GROUP BY

```
SELECT
age,
COUNT(*) AS number_of_students
FROM students
GROUP BY age;
```

Meaning: Put students with the same age into groups, then count each group.

5.3 HAVING

```
SELECT
age,
COUNT(*) AS number_of_students
FROM students
GROUP BY age
HAVING COUNT(*) > 1;
```

Remember: WHERE filters rows; HAVING filters groups.

6. DDL — Build and Change Structure

6.1 ALTER TABLE

```
-- Add
ALTER TABLE students
ADD phone VARCHAR(20);

-- Rename
ALTER TABLE students
RENAME COLUMN phone TO phone_number;

-- Change type
ALTER TABLE students
MODIFY phone_number VARCHAR(30);

-- Remove
ALTER TABLE students
DROP COLUMN phone_number;
```

ALTER TABLE means “change the design of an existing table.”

6.2 DROP vs TRUNCATE vs DELETE

Command	What it removes	Table structure remains?
DELETE	Selected rows, or all rows if no WHERE	Yes
TRUNCATE	All rows	Yes
DROP	The table itself and its data	No

Safety: DELETE, TRUNCATE and DROP can be destructive. Practice them on lab tables, not original sample data.

7. Constraints — Rules for Data

A constraint is a rule the database enforces so that invalid or unwanted data is harder to store.

Constraint	Simple meaning
PRIMARY KEY	Unique identity of a row
AUTO_INCREMENT	Let MySQL generate the next numeric ID
FOREIGN KEY	Connect a value to a valid value in another table

Constraint	Simple meaning
UNIQUE	Do not allow duplicate values
NOT NULL	A value is required
CHECK	A condition must be satisfied
DEFAULT	Use a value automatically when one is not supplied

8. DML — Work with Data

8.1 INSERT

```
INSERT INTO students
(student_name, age, course, marks, email)
VALUES
('Rahul', 20, 'BCA', 78, 'rahul@gmail.com');
```

Meaning: Add a new row.

8.2 INSERT multiple rows

```
INSERT INTO students
(student_name, age, course, marks, email)
VALUES
('Anita', 21, 'BCA', 88, 'anita@gmail.com'),
('Kiran', 19, 'BCA', 65, 'kiran@gmail.com'),
('Priya', 22, 'BCA', 92, 'priya@gmail.com');
```

8.3 UPDATE

```
-- First check the target
SELECT *
FROM students
WHERE student_id = 1;

-- Then change it
UPDATE students
SET marks = 85
WHERE student_id = 1;
```

Critical rule: Before UPDATE, run the matching SELECT. An UPDATE without a WHERE clause can change every row.

8.4 DELETE

```
SELECT *
FROM students
WHERE student_id = 7;

DELETE FROM students
WHERE student_id = 7;
```

The same safety rule applies to DELETE. Without a WHERE clause, every row can be deleted.

8.5 REPLACE

```
REPLACE INTO students
(student_id, student_name, age, course, marks, email)
VALUES
(1, 'Rahul Kumar', 20, 'BCA', 90, 'rahul@gmail.com');
```

Important: REPLACE is not simply another spelling of UPDATE. With a conflicting unique/primary key, MySQL's REPLACE behavior is essentially delete-and-insert.

9. Foreign Keys and Relationships

9.1 Create departments

```
CREATE TABLE departments (  
  department_id INT PRIMARY KEY AUTO_INCREMENT,  
  department_name VARCHAR(100) NOT NULL UNIQUE  
);  
  
INSERT INTO departments (department_name)  
VALUES  
  ('Computer Science'),  
  ('Data Science'),  
  ('Commerce');
```

9.2 Connect students to departments

```
ALTER TABLE students  
ADD department_id INT;  
  
ALTER TABLE students  
ADD CONSTRAINT fk_students_department  
FOREIGN KEY (department_id)  
REFERENCES departments(department_id);
```

Layman meaning: A student can point to a department. The foreign key tells MySQL that the referenced department must exist.

10. JOINS — Bring Tables Together

10.1 INNER JOIN

```
SELECT  
  s.student_name,  
  s.marks,  
  d.department_name  
FROM students AS s  
INNER JOIN departments AS d  
ON s.department_id = d.department_id;
```

Meaning: Return students where a matching department exists.

10.2 LEFT JOIN

```
SELECT  
  s.student_name,  
  d.department_name  
FROM students AS s  
LEFT JOIN departments AS d  
ON s.department_id = d.department_id;
```

Meaning: Keep every student, even if no matching department exists.

10.3 Find missing matches

```
SELECT  
  s.student_name  
FROM students AS s  
LEFT JOIN departments AS d  
ON s.department_id = d.department_id  
WHERE d.department_id IS NULL;
```

This is a common pattern for finding rows in the left table that have no matching row in the right table.

10.4 SELF JOIN

```
SELECT  
  s1.student_name AS student_1,  
  s2.student_name AS student_2,  
  s1.age  
FROM students AS s1
```

```
JOIN students AS s2
ON s1.age = s2.age
AND s1.student_id < s2.student_id;
```

A self join joins a table to itself. Aliases such as **s1** and **s2** make the two roles clear.

10.5 ON vs USING

```
-- Explicit relationship
SELECT *
FROM students AS s
JOIN departments AS d
ON s.department_id = d.department_id;
```

Use **ON** when you want to explicitly state the relationship. **USING(column)** is available when both tables have the same join-column name.

11. Subqueries

11.1 Subquery — a query inside a query

```
SELECT
student_name,
marks
FROM students
WHERE marks > (
SELECT AVG(marks)
FROM students
);
```

The inner query calculates the average. The outer query uses that answer to find students above average.

11.2 IN

```
SELECT *
FROM students
WHERE department_id IN (1, 2);
```

IN means the value is one of the listed values or returned values.

11.3 EXISTS

```
SELECT
d.department_id,
d.department_name
FROM departments AS d
WHERE EXISTS (
SELECT 1
FROM students AS s
WHERE s.department_id = d.department_id
);
```

EXISTS asks whether at least one matching row exists.

11.4 NOT EXISTS

```
SELECT
d.department_id,
d.department_name
FROM departments AS d
WHERE NOT EXISTS (
SELECT 1
FROM students AS s
WHERE s.department_id = d.department_id
);
```

NOT EXISTS asks whether no matching row exists.

12. NULL — Missing or Unknown

NULL is not the same as zero, an empty string, or false. Treat it as an unknown/missing value.

```
-- Correct
SELECT *
FROM students
WHERE department_id IS NULL;

-- Correct
SELECT *
FROM students
WHERE department_id IS NOT NULL;
```

Do not normally write **column = NULL** to test for NULL. Use **IS NULL** or **IS NOT NULL**.

13. CTE — WITH

```
WITH high_scorers AS (
  SELECT
    student_name,
    marks
  FROM students
  WHERE marks >= 80
)
SELECT *
FROM high_scorers;
```

A CTE gives a name to a query result so the main query is easier to read. Think of it as a named temporary building block for the query.

13.1 Multiple CTEs

```
WITH student_stats AS (
  SELECT
    COUNT(*) AS total_students,
    AVG(marks) AS average_marks
  FROM students
),
top_students AS (
  SELECT COUNT(*) AS top_student_count
  FROM students
  WHERE marks >= 80
)
SELECT *
FROM student_stats
CROSS JOIN top_students;
```

13.2 Recursive CTE

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1
  FROM numbers
  WHERE n < 10
)
SELECT *
FROM numbers;
```

A recursive CTE repeatedly builds a result from a previous result. It can be used for sequences, dates, and hierarchical data.

14. Window Functions

A window function calculates across related rows while normally keeping the individual rows visible. This is different from GROUP BY, which normally collapses rows into groups.

14.1 ROW_NUMBER

```
SELECT
  student_name,
  marks,
  ROW_NUMBER() OVER (ORDER BY marks DESC) AS row_number_value
FROM students;
```

Every row gets a unique sequence number.

14.2 RANK and DENSE_RANK

```
SELECT
  student_name,
  marks,
  RANK() OVER (ORDER BY marks DESC) AS rank_value,
  DENSE_RANK() OVER (ORDER BY marks DESC) AS dense_rank_value
FROM students;
```

Function	Tie behavior
ROW_NUMBER	Each row gets a different number
RANK	Ties share rank; later ranks can have gaps
DENSE_RANK	Ties share rank; later ranks do not have gaps

14.3 PARTITION BY

```
SELECT
  student_name,
  department_id,
  marks,
  RANK() OVER (
    PARTITION BY department_id
    ORDER BY marks DESC
  ) AS department_rank
FROM students;
```

PARTITION BY means perform the window calculation separately for each group.

14.4 LAG and LEAD

```
SELECT
  student_name,
  marks,
  LAG(marks) OVER (ORDER BY student_id) AS previous_marks,
  LEAD(marks) OVER (ORDER BY student_id) AS next_marks
FROM students;
```

LAG looks backward; **LEAD** looks forward.

15. Views, Indexes and EXPLAIN

15.1 VIEW

```
CREATE VIEW student_summary AS
SELECT
  s.student_id,
  s.student_name,
  s.marks,
  d.department_name
FROM students AS s
LEFT JOIN departments AS d
ON s.department_id = d.department_id;

SELECT *
FROM student_summary;
```

A view is a saved query that can be queried like a table. It is useful for reusable reporting logic.

15.2 INDEX

```
CREATE INDEX idx_students_marks
ON students(marks);

SHOW INDEX FROM students;
```

An index is similar to the index of a textbook: it can help MySQL locate data without checking every row.

15.3 EXPLAIN

```
EXPLAIN
SELECT *
FROM students
WHERE marks = 85;
```

EXPLAIN asks MySQL how it plans to execute a query. In the session plan, students focus on fields such as **type**, **key**, **rows**, and **Extra**.

16. Transactions — Save or Undo Changes

16.1 START TRANSACTION + ROLLBACK

```
START TRANSACTION;

UPDATE students
SET marks = 100
WHERE student_id = 1;

SELECT *
FROM students
WHERE student_id = 1;

ROLLBACK;

SELECT *
FROM students
WHERE student_id = 1;
```

The idea is: **start** a controlled set of changes → make a change → inspect it → **ROLLBACK** if you want to undo it.

16.2 COMMIT

```
START TRANSACTION;

UPDATE students
SET marks = 90
WHERE student_id = 1;

COMMIT;
```

COMMIT means: “Keep these transaction changes.”

16.3 SAVEPOINT

```
START TRANSACTION;

UPDATE students
SET marks = 95
WHERE student_id = 1;

SAVEPOINT first_change;

UPDATE students
SET marks = 50
WHERE student_id = 2;
```

```
ROLLBACK TO SAVEPOINT first_change;

COMMIT;
```

A SAVEPOINT is like a bookmark inside a transaction. You can roll back to that point without necessarily undoing everything before it.

Important MySQL note: DDL such as CREATE TABLE can cause implicit commits. Do not assume every DDL operation can be undone with ROLLBACK.

17. Moving from the Custom Database to employees and sakila

Once students understand the custom database, repeat the same concepts using the two databases in the supplied module plan.

Concept	Custom database	employees / sakila practice
SELECT / WHERE	students	employees / film
GROUP BY	students by age	employees by gender / film by rating
JOIN	students + departments	employees + dept_emp + departments / customer + rental
Subquery	marks above average	salary or film length above average
EXISTS	departments with students	customers with rentals
CTE	high_scorers	employee or film summaries
Window functions	student ranking	salary / payment analysis
Views	student_summary	department or film summary
EXPLAIN / index	students.marks	appropriate employees / sakila columns

18. Final Combined Practice

18.1 Average marks by department

```
SELECT
  d.department_name,
  COUNT(s.student_id) AS student_count,
  AVG(s.marks) AS average_marks,
  MAX(s.marks) AS highest_marks
FROM departments AS d
LEFT JOIN students AS s
ON d.department_id = s.department_id
GROUP BY d.department_id, d.department_name
ORDER BY average_marks DESC;
```

This combines **LEFT JOIN + GROUP BY + aggregate functions + ORDER BY**.

18.2 Top 2 students in each department

```
WITH ranked_students AS (
  SELECT
    s.student_name,
    s.marks,
    d.department_name,
    ROW_NUMBER() OVER (
      PARTITION BY s.department_id
      ORDER BY s.marks DESC
    ) AS student_rank
  FROM students AS s
  JOIN departments AS d
  ON s.department_id = d.department_id
```

```

)
SELECT
student_name,
marks,
department_name,
student_rank
FROM ranked_students
WHERE student_rank <= 2
ORDER BY department_name, student_rank;

```

This combines **CTE + JOIN + ROW_NUMBER + PARTITION BY + ORDER BY + WHERE**.

19. Beginner SQL Cheat Sheet

Keyword / Command	Think of it as
CREATE DATABASE	Build a new cupboard
USE	Choose the cupboard
CREATE TABLE	Build a new drawer/form
SELECT	Show me
FROM	Take it from
WHERE	Only when
AS	Give this result a temporary name
DISTINCT	Remove duplicate results
ORDER BY	Sort it
LIMIT	Show only this many
GROUP BY	Make groups
HAVING	Filter groups
INSERT	Add a row
UPDATE	Change existing data
DELETE	Remove rows
ALTER	Change table structure
DROP	Remove the object itself
TRUNCATE	Remove all rows but keep structure
JOIN	Connect tables
ON	Explain the connection
IN	Matches one of these values
EXISTS	At least one matching row exists
WITH	Name a query result
OVER	Define a window for a window function
PARTITION BY	Do the window calculation separately by group
COMMIT	Save transaction changes
ROLLBACK	Undo transaction changes

Keyword / Command	Think of it as
SAVEPOINT	Create a rollback bookmark
EXPLAIN	Show the planned execution approach
CREATE INDEX	Create a lookup structure to help searching

20. Student Practice Checklist

- Create the database and table without copying the answer.
- Insert five students.
- Select all rows, then select only two columns.
- Use AS to rename two output columns.
- Filter marks using WHERE.
- Combine conditions using AND and OR.
- Sort marks in ascending and descending order.
- Count students and calculate average marks.
- Group students by age.
- Use HAVING to filter a group.
- Update one student's marks safely.
- Delete one test row safely.
- Create the departments table.
- Connect students to departments with a foreign key.
- Write an INNER JOIN.
- Write a LEFT JOIN.
- Find students with no department.
- Find students above the average using a subquery.
- Use EXISTS and NOT EXISTS.
- Create a CTE.
- Rank students with ROW_NUMBER, RANK and DENSE_RANK.
- Rank students separately by department.
- Use LAG and LEAD.
- Create a view.
- Create an index and inspect it.
- Run EXPLAIN.
- Test ROLLBACK, COMMIT and SAVEPOINT.
- Write the final top-2-per-department query.

21. Safety Rules Students Must Remember

- **Before UPDATE:** run SELECT with the same WHERE condition.

- **Before DELETE:** run SELECT with the same WHERE condition.
- **Do not experiment with DROP/TRUNCATE on the original sample databases.**
- **NULL is not zero.** Use IS NULL / IS NOT NULL.
- **WHERE filters rows; HAVING filters groups.**
- **JOIN needs a clear relationship.** Understand the ON condition before running it.
- **Transactions do not magically protect every operation.** MySQL DDL can cause implicit commits.
- **Change one thing at a time while learning.** It makes cause and effect obvious.

End of Student Notes — MySQL Module 1